InsightAgent — Technical Report

Module: XJCO3011 Web Services and Web Data Assessment: Coursework 1 — Individual Web Services API Development Project

GitHub: [PLACEHOLDER] API documentation PDF: [PLACEHOLDER] Slides/visuals: [PLACEHOLDER]

1. Project Overview

InsightAgent is an enterprise sales data Web API built on the Classic Models relational dataset and extended with a Qwen-powered self-loop analysis agent. A client can submit a natural-language business question such as *"Which office had the lowest revenue last year, and which sales representatives work there?"* and receive a grounded answer derived from live database queries rather than from hard-coded SQL templates. The system uses the Classic Models sample database as its relational backbone. This is a publicly available MySQL sample dataset distributed as a SQL dump through the MySQL Tutorial sample-database resource, covering offices, employees, customers, orders, order details, products, product lines, and payments; this structure makes it appropriate for transactional and analytical enterprise-style questions and for natural-language interfaces over relational data (MySQL Tutorial, n.d.; Affolter, Stockinger and Bernstein, 2019; Liu *et al.*, 2026). The language-model component uses a Model Studio/OpenAI-compatible Qwen endpoint for reasoning and orchestration (Alibaba Cloud, 2026).

The project exposes nineteen REST endpoints across five capability groups: reference-data retrieval for employees and products, full CRUD for saved analytical queries, deterministic analytics endpoints, direct skill endpoints for schema inspection and sandboxed Python execution, and an orchestration endpoint that drives the LLM self-loop. FastAPI was selected because it combines lightweight routing with built-in OpenAPI generation, validation, and interactive documentation (FastAPI, n.d.a; FastAPI, n.d.b). Docker Compose is used so that the API service and MySQL database can be reproduced from a clean environment with one command, while container-to-container communication uses the default service-name network provided by Compose (Docker, n.d.a; Docker, n.d.b).

This design addresses two goals simultaneously. First, it satisfies the coursework baseline of a database-backed data API with validation, authentication, structured JSON responses, and reproducible deployment. Second, it demonstrates a modern AI-assisted analytics workflow in which the LLM is not treated as a direct source of business facts, but as a planner and code generator that must operate through explicit tools and verifiable query outputs.

2. API Functionality and Data Model Coverage

The API was designed to demonstrate both conventional engineering discipline and analytical flexibility. At the standard API level, it provides reference-data retrieval for `employees` and `products`, complete CRUD for `saved_queries`, health/authentication endpoints, and deterministic analytics routes such as store-sales summary, product ranking, employee performance, and sales trends. This combination shows that the service can support both predictable operational queries and more open-ended analytical requests.

The relational backbone remains close to the Classic Models schema: `employees`, `offices`, `customers`, `orders`, `orderdetails`, `products`, `productlines`, and `payments`. These are extended with `saved_queries`, `analysis_logs`, and `agent_turns`. The added tables store the original natural-language question, generated analysis code, final summaries, and the turn-by-turn tool trace needed for auditability. Keeping the core schema recognisably relational was important because the coursework expects at least one genuine CRUD-backed data model and appropriate JSON responses over a database connection. Using a sample dataset already distributed as a MySQL dump also removed unnecessary data-conversion work and kept the implementation aligned with relational best practice; it also made it easier to support schema-aware natural-language analysis over a realistic enterprise schema (MySQL Tutorial, n.d.; Pazos *et al.*, 2016; Kim and Okubo, 2024).

Pre-built analytics endpoints are not redundant with the LLM agent. Instead, they act as a low-latency baseline proving that the underlying SQL and API design are sound even without AI assistance. The agent pathway then becomes an additional intelligent layer rather than a substitute for ordinary API quality.

3. Technology Stack Justification

Why Python? Python was selected because the analytical core relies on first-class support for `pandas`, `numpy`, LLM client libraries, and the standard-library `ast` module used for static code inspection (Python Software Foundation, 2024). Recreating the same sandboxed analysis pipeline in Java or Go would require significantly more scaffolding.

Why FastAPI instead of Django or Flask? Django introduces framework overhead that is unnecessary for an API-only coursework project, while Flask would require additional tooling to reach the same level of schema generation and validation. FastAPI provides a practical middle ground with OpenAPI-based automatic documentation and interactive Swagger UI by default (FastAPI, n.d.a; FastAPI, n.d.b).

Why MySQL instead of PostgreSQL or MongoDB? The Classic Models dataset is already distributed as a MySQL sample database (MySQL Tutorial, n.d.). In addition, the application depends on strongly relational joins between employees, offices, products, orders, customers, and payments; a document store would weaken rather than simplify that design.

Why a self-loop agent instead of a single LLM call? A one-shot LLM answer is unreliable for multi-step business questions because the model cannot know the exact schema in advance and may hallucinate plausible but false numbers. The self-loop pattern — `observe_schema` → `run_python_analysis` → `final_answer` — forces the model to ground its output in actual query results.

Component	Choice	Justification
API framework	FastAPI (Python 3.11)	Type-driven validation, OpenAPI/Swagger generation, and efficient routing suitable for a data API (FastAPI, n.d.a; FastAPI, n.d.b).
Database	MySQL 8	Strong fit for the Classic Models dump and relational analytics workloads, with row-level locking and nonlocking consistent reads in InnoDB (MySQL Tutorial, n.d.; MySQL, n.d.).
ORM / data access	SQLAlchemy 2.0	Explicit session management and a clear split between Core and ORM make the dual-account pattern straightforward to implement (SQLAlchemy, 2026).
LLM provider	Qwen via Alibaba Model Studio	OpenAI-compatible API surface allows provider changes via configuration rather than orchestration rewrites (Alibaba Cloud, 2026).
Execution model	Subprocess sandbox	Isolates untrusted generated code from the main API process and simplifies timeout and output capture.
Containerisation	Docker + Docker Compose	Removes environment drift and provides service-name networking for the API and database containers (Docker, n.d.a; Docker, n.d.b).
Testing	pytest + pytest-rerunfailures	Supports API, unit, and end-to-end tests; reruns mitigate transient model-provider instability (pytest-rerunfailures, 2025).

Table 1: Core stack choices and their rationale.

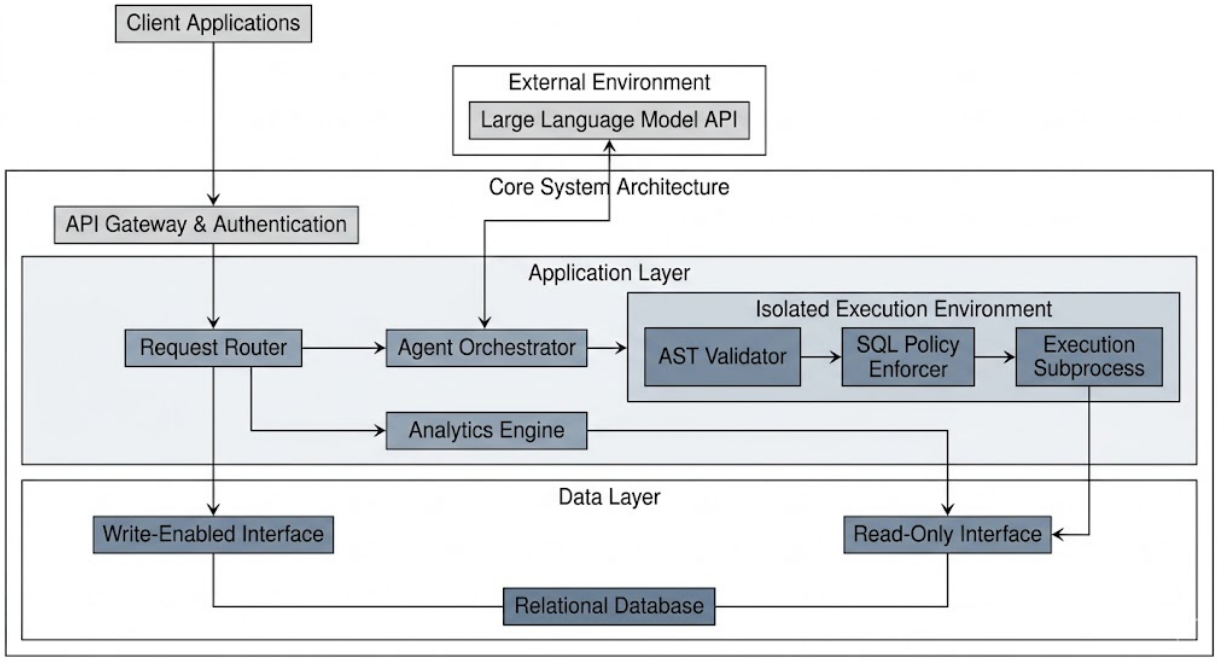

Figure 1: System architecture. The API server, external LLM, isolated execution environment, and read-write/read-only data paths are separated explicitly.

4. System Architecture and Database Design

The API is organised into four layers. The **transport layer** provides HTTP access and API-key authentication. The **route layer** groups endpoints by domain: employees, products, saved queries, analytics, and skills. The **service layer** isolates business logic from HTTP concerns, with dedicated modules for analytics, skills, and agent orchestration. The **data layer** contains SQLAlchemy-backed access paths, deliberately split into read-write and read-only channels.

This separation has two practical benefits. First, the agent logic can be tested independently of routing, reducing coupling and making the orchestrator easier to reason about. Second, the execution sandbox can evolve without route-level rewrites because it is treated as a service component rather than as controller code. The database uses the Classic Models schema and extends it with custom audit tables. `saved_queries` stores the original question, generated code, and saved summary. `analysis_logs` and `agent_turns` preserve per-iteration behaviour for debugging and viva defensibility.

Concurrency is delegated primarily to MySQL and SQLAlchemy. InnoDB performs row-level locking and nonlocking consistent reads by default, which makes it appropriate for concurrent read/write access in this kind of transactional service (MySQL, n.d.). The application layer mirrors this with separate engine/session factories for read-write and read-only access.

4.1 Natural-Language Analysis Path

The natural-language endpoint follows a strict pipeline. A client submits a question; the orchestrator injects the system policy and asks the remote LLM to decide the next action; the model first calls `observe_schema`, then emits Python analysis code, and finally synthesises a textual answer from verified tool outputs. The answer, generated code, number of iterations, and tool trace are persisted so that the result is reproducible and reviewable after the fact.

This architecture keeps the LLM away from unrestricted database access. The model never receives a write-capable

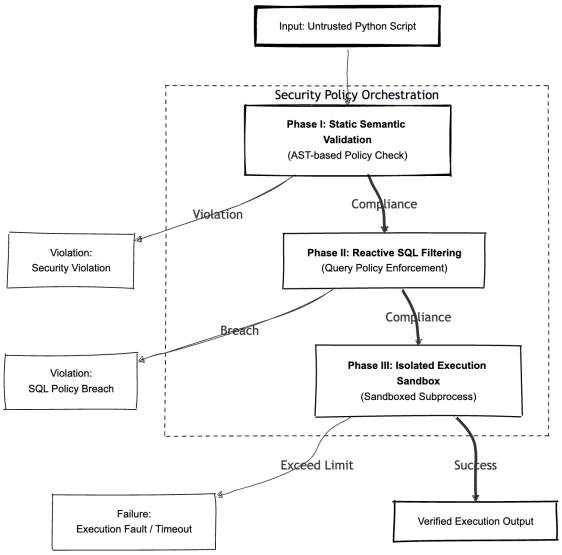

Figure 2: Three-phase security policy orchestration for untrusted Python analysis code.

driver and never answers directly from parametric memory. It must work through inspectable tools, which makes the final response easier to trust and easier to defend in the viva.

The route surface was intentionally split into ordinary business endpoints and advanced analytical endpoints. The former are easy to demonstrate during the oral examination because they expose conventional CRUD and read-only retrieval behaviour; the latter show how the same data layer can support richer analytical capabilities without introducing a second application. This distinction also helped keep responsibilities clear in the codebase: ordinary routes remain thin wrappers over services, whereas the agent pathway is treated as an orchestration problem with explicit tool contracts.

5. Key Design Decisions

5.1 Dual Database Accounts

Rather than relying on a single database user, the system uses two accounts. `app_rw` is used by CRUD routes and has `SELECT/INSERT/UPDATE/DELETE` privileges. `app_ro` is used by the schema reader, analytics service, and sandboxed analysis skill, and has `SELECT`-only privileges. This creates infrastructure-level defence in depth: even if generated Python code bypassed application checks and reached the database helper, the underlying account would still prevent writes.

5.2 Three-Phase Security Policy Orchestration

The most security-sensitive capability in the system is execution of model-generated Python code. To reduce risk, every submitted script passes through three phases before it is allowed to run.

Phase I: static semantic validation. Python's `ast` module parses the code before execution and rejects dangerous imports such as `os`, `sys`, `subprocess`, `socket`, `requests`, and `pathlib`, as well as direct calls to `open()`, `eval()`, `exec()`, `compile()`, and `__import__()` (Python Software Foundation, 2024).

Phase II: reactive SQL filtering. The sandbox exposes a helper named `read_sql()` rather than a raw connection object. That helper inspects every SQL string and blocks non-read operations such as `INSERT`, `UPDATE`, `DELETE`, `DROP`, and `CREATE`. This second gate exists because AST checks alone cannot reason about SQL embedded inside Python strings.

Phase III: isolated execution sandbox. Approved code is executed in a child subprocess with a restricted namespace, controlled resources, and a fixed timeout. This contains runtime faults and helps keep the main API process stable even when generated code fails.

6. LLM Agent Design

The Agent Orchestrator implements a ReAct-style self-loop in which the model alternates between reasoning and tool use until it can produce a final synthesis. The agent has access to three effective actions: inspect the schema, execute analysis code, and terminate with a final answer. This keeps the tool surface small enough to audit while still supporting flexible analysis.

The behavioural policy is deliberately opinionated. The agent must invoke `observe_schema` before generating analysis code. If `run_python_analysis` returns an execution error or a security block, the model is instructed to read the message carefully, revise the code, and try again rather than repeat the same action. Near the end of the iteration budget, the orchestrator forces a terminal `final_answer` step. Each turn is stored in `agent_turns`, making the system's reasoning trace inspectable via API endpoints.

This design matters for coursework quality because it demonstrates not only that the model can answer questions, but that the answer is recoverable, auditable, and rooted in actual tool output. The analysis path therefore behaves more like an instrumented analytics assistant than a generic chatbot.

7. Testing Strategy, Challenges, and Lessons Learned

The project includes ninety-three tests spread across six test files. The test plan combines API contract validation, security-focused negative tests, multistep agent tests, anomaly-detection evaluation, and LLM-as-a-judge assessments where exact string matching would be brittle. The rerun plugin was used to manage intermittent upstream model-provider instability during repeated test runs (pytest-rerunfailures, 2025).

Three principles shaped the test strategy. First, tests assert the contract rather than the internal implementation, so CRUD tests focus on status codes and response shapes rather than ORM details. Second, each major endpoint has explicit negative coverage for authentication failures, invalid parameters, missing resources, and policy violations. Third, natural-language outputs are evaluated semantically rather than literally: an AI Judge receives the original question, the final answer, the tool trace, and a human-written criterion, then returns a reasoned yes/no verdict.

Several implementation challenges shaped the final design. Ambiguous-column SQL errors were solved by enforcing aliases in multi-table joins. Empty-result loops were reduced by treating empty output as a legitimate analytical

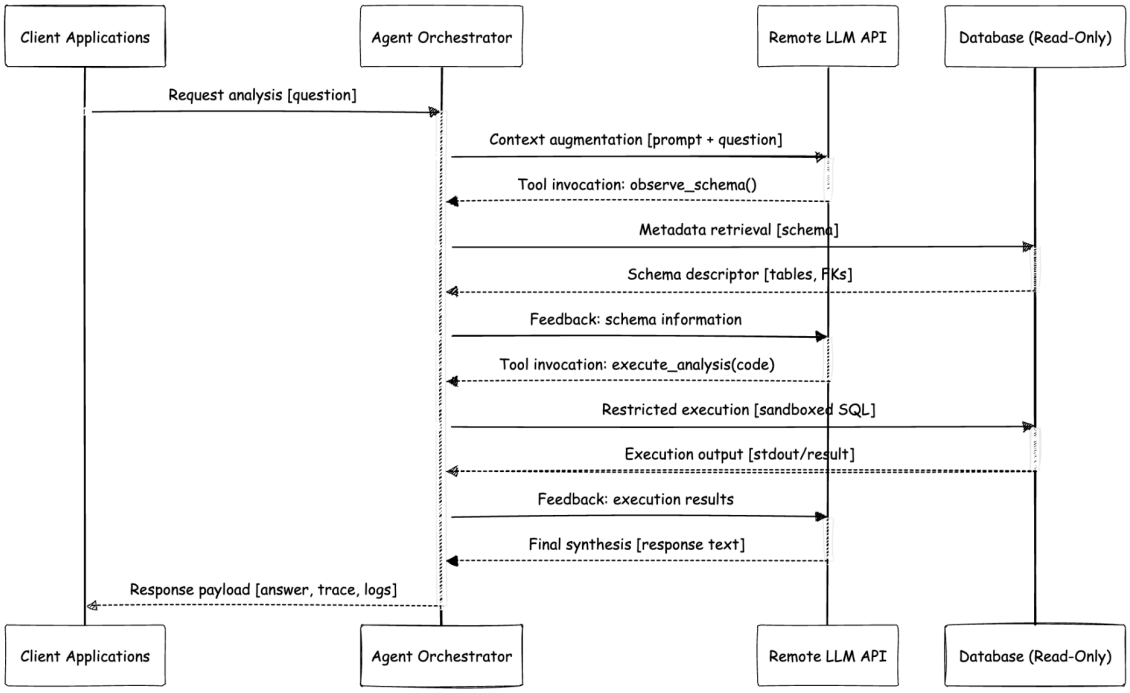

Figure 3: Agent sequence flow. The orchestrator first augments context, then iteratively alternates between schema observation, analysis execution, and response synthesis.

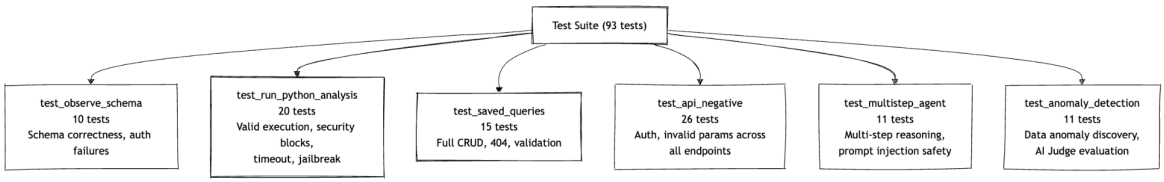

Figure 4: Test-suite structure covering schema inspection, sandbox execution, CRUD, negative API cases, multistep agent reasoning, and anomaly detection.

finding rather than as a silent failure. Security design also evolved: an early AST-only version looked safe but failed to prevent destructive SQL embedded inside otherwise valid Python. Adding reactive SQL filtering closed that gap and reinforced a core lesson — security must be layered, not single-gated.

A second lesson was that agent prompting is iterative engineering rather than one-off writing. Early prompts caused the agent to repeat broken code. Explicit instructions to read error messages, revise code, and terminate when the iteration budget is nearly exhausted materially improved completion rates. A related lesson was that observability matters: without saving the tool trace, many failure modes would have been much harder to diagnose in a principled way.

The testing suite was organised not simply by file or module, but by risk. Separate groups target schema inspection, sandbox execution success and failure paths, saved-query CRUD, negative API behaviour across all routes, multistep agent reasoning, and anomaly-detection scenarios. This distribution ensured that the AI-specific parts of the system were not tested in isolation but in combination with the conventional API contract expected of a coursework-grade web service. As a result, the final implementation can be defended both as a software-engineering artifact and as an experimental LLM-enabled analytics system.

8. Documentation, Deployment, Submission Readiness, and Future Work

The project was packaged so that a marker can reproduce it with minimal setup. The underlying business data comes from the Classic Models sample database, a public MySQL sample schema and dataset used here as the source of employees, offices, customers, orders, products, order details, and payments (MySQL Tutorial, n.d.). Docker Compose starts two containers — the FastAPI application and a MySQL instance seeded from the Classic Models dump and custom migration scripts. Swagger UI and OpenAPI are generated automatically by FastAPI, reducing documentation drift between implementation and the described contract (FastAPI, n.d.b). The written API documentation complements Swagger with explicit authentication behaviour, request/response shapes, and skill-specific error semantics.

From a deployment perspective, Compose provides a simple reproducibility story. The MySQL container is initialised with the sample dump plus custom tables for saved queries and analysis logs. The FastAPI container waits for database readiness, then starts with environment-based configuration for the read-write account, read-only

account, and remote LLM API key. Compose networking makes each service reachable by service name inside the application network (Docker, n.d.a).

Another submission-oriented design choice was alignment between implementation and evidence. The same route groups exposed in the live API are mirrored in the written API documentation and in the structure of the oral presentation. This reduces the risk of "documentation drift", where the report, the endpoint list, and the running server describe different systems. In a coursework setting, that consistency is particularly valuable because examiners review the repository, API documentation, technical report, and oral demonstration as a package rather than as isolated artifacts.

Current limitations remain. The agent is stateless across requests, there is no production-grade rate limiting, the system depends on an external Qwen API, and `read_sql()` imposes a practical row cap. Future extensions could add multi-turn memory, streaming responses, provider abstraction for multiple LLM backends, scoped role-based API keys, vector retrieval over past analyses, and stronger OS-level isolation for the execution worker.

Taken together, the final design aims to balance conventional web-service engineering with a carefully bounded AI-assisted analytics layer. The CRUD routes, OpenAPI contract, Docker deployment, and relational persistence satisfy the expectations of a serious data API, while the agent loop, schema tool, and sandboxed execution path extend that baseline into a more ambitious analytical assistant. The strongest claim of the project is therefore not simply that it uses an LLM, but that it integrates an LLM into a structured architecture with explicit interfaces, controlled privileges, reproducible deployment, and measurable testing.

References

Alibaba Cloud. 2026. *What is Alibaba Cloud Model Studio*. [Online]. [Accessed 20 April 2026]. Available from: https://www.alibabacloud.com/help/en/model-studio/what-is-model-studio

Docker. n.d.a. *Networking in Compose*. [Online]. [Accessed 20 April 2026]. Available from: https://docs.docker.com/compose/how-tos/networking/

Docker. n.d.b. *Define and manage networks in Docker Compose*. [Online]. [Accessed 20 April 2026]. Available from: https://docs.docker.com/reference/compose-file/networks/

FastAPI. n.d.a. *Features*. [Online]. [Accessed 20 April 2026]. Available from: https://fastapi.tiangolo.com/features/

FastAPI. n.d.b. *Metadata and docs URLs*. [Online]. [Accessed 20 April 2026]. Available from: https://fastapi.tiangolo.com/tutorial/metadata/

Affolter, K., Stockinger, K. and Bernstein, A. 2019. 'A comparative survey of recent natural language interfaces for databases'. *The VLDB Journal*. 28(5), pp. 793–819. Available from: https://doi.org/10.1007/s00778-019-00567-8

Katsogiannis-Meimarakis, G. and Koutrika, G. 2023. 'A survey on deep learning approaches for text-to-SQL'. *The VLDB Journal*. 32(4), pp. 905–936. Available from: https://doi.org/10.1007/s00778-022-00776-8

Kim, J.-D. and Okubo, K. 2024. 'Customizing GPT for natural language dialogue interface in database access'. *Genomics & Informatics*. 22, 22. Available from: https://doi.org/10.1186/s44342-024-00020-5

Li, M.Q. and Fung, B.C.M. 2025. 'Security concerns for large language models: A survey'. *Journal of Information Security and Applications*. 95, 104284. Available from: https://doi.org/10.1016/j.jisa.2025.104284

Liu, M., Wang, X., Xu, J. et al. 2026. 'A systematic review of natural language interfaces for databases'. *Frontiers of Computer Science*. 20, 2011623. Available from: https://doi.org/10.1007/s11704-025-50592-w

MySQL. n.d. *InnoDB transaction model*. [Online]. [Accessed 20 April 2026]. Available from: https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-model.html

MySQL Tutorial. n.d. *MySQL sample database*. [Online]. [Accessed 20 April 2026]. Available from: https://www.mysqltutorial.org/getting-started-with-mysql/mysql-sample-database/

Pazos, R.A., Aguirre, M.A., González, J.J., Martínez, J.A., Pérez, J., Esquivel, A., Alor-Hernández, G. and Sánchez-Cervantes, J.L. 2016. 'Comparative study on the customization of natural language interfaces to databases'. *SpringerPlus*. 5, 553. Available from: https://doi.org/10.1186/s40064-016-2164-y

pytest-rerunfailures. 2025. *pytest-rerunfailures*. [Online]. [Accessed 20 April 2026]. Available from: https://pypi.org/project/pytest-rerunfailures/

Python Software Foundation. 2024. *ast — Abstract Syntax Trees*. [Online]. [Accessed 20 April 2026]. Available from: https://docs.python.org/3.12/library/ast.html

SQLAlchemy. 2026. *SQLAlchemy 2.0 documentation: Overview*. [Online]. [Accessed 20 April 2026]. Available from: https://docs.sqlalchemy.org/20/intro.html

Appendix A. Generative AI declaration and evidence

This appendix explains how Generative AI tools were used at different stages of the project. The screenshots are included as representative evidence rather than substitutes for technical justification. In each case, the outputs were treated as proposals to be evaluated, tested, revised, and, where necessary, rejected or constrained.

A1. ChatGPT — architecture alternatives exploration

Role in the project. ChatGPT was used during the early design stage to explore high-level architectural alternatives for InsightAgent. The purpose was not to generate code immediately, but to compare several possible system structures for an enterprise analytics API with a natural-language analysis pathway.

Representative prompt. Compare six architecture options for an enterprise analytics API with natural-language analysis: (1) standard CRUD-only API, (2) CRUD plus pre-built analytics endpoints, (3) one-shot LLM answer generation, (4) self-loop LLM agent with tools/skills, (5) direct text-to-SQL generation, and (6) a schema-observation plus Python-analysis workflow. For each option, explain its strengths, weaknesses, implementation complexity, reliability, security implications, and suitability for a university technical project.

Summary of the response. The response compared each option systematically and concluded that the strongest overall architecture was a *self-loop LLM agent built around two core tools:* `observe_schema` *and* `run_python_analysis`. It argued that this combination balanced architectural structure, analytical flexibility, and a stronger security story than either one-shot prompting or direct text-to-SQL alone.

My reflection, judgement, and actions taken. This interaction helped me move beyond a generic CRUD or one-shot LLM design and think in terms of architectural trade-offs. After reading the response, I decided to adopt a hybrid structure: standard FastAPI CRUD and analytics endpoints for the conventional API layer, plus a separate advanced analysis pathway based on a self-loop agent, with `observe_schema` for database inspection and `run_python_analysis` for controlled analytical execution. I did not adopt the response blindly; I used it as a design comparison and then selected only the parts that best matched the coursework scope, technical ambition, and security requirements.

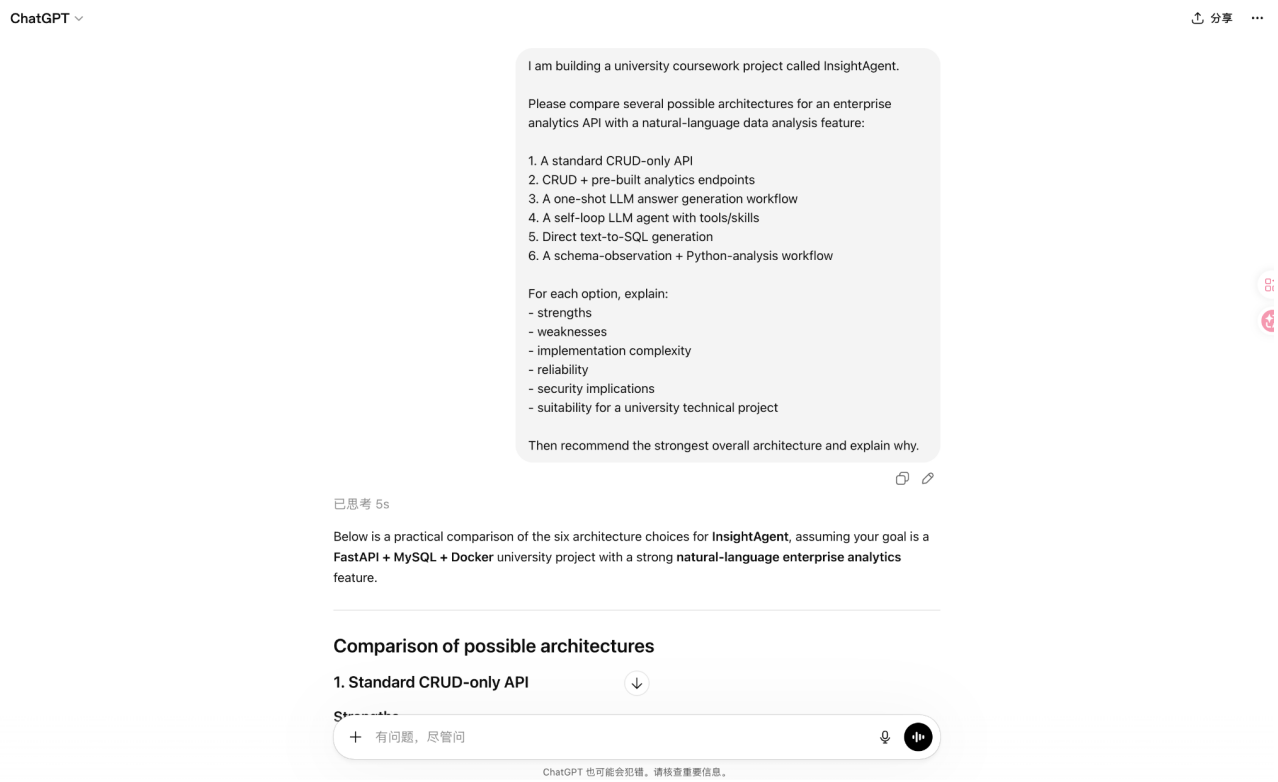

Figure 5: ChatGPT-assisted exploration of alternative system architectures for InsightAgent.

A2. ChatGPT — security architecture and defence-in-depth exploration

Role in the project. ChatGPT was also used to explore secure execution strategies for untrusted Python code generated by an LLM. This was a critical design problem because the project includes a `run_python_analysis` skill that executes model-generated analysis code.

Representative prompt. Compare unrestricted `exec`, filtered namespaces, AST-based static validation, SQL policy filtering, subprocess isolation, Docker/container sandboxing, read-only database accounts, and defence-in-depth combinations. For each approach, explain which threats it mitigates, what risks remain, the implementation difficulty, and whether it is suitable for a university project. Then propose a practical multi-layer security design for a FastAPI + MySQL project.

Summary of the response. The response explained the strengths, weaknesses, and residual risks of each security approach. It identified unrestricted execution and filtered namespaces as weak or incomplete, and recommended a *multi-layer defence-in-depth execution model* combining AST-based validation, controlled SQL policy enforcement, subprocess or container isolation, read-only database credentials, timeout and resource limits, and audit logging.

My reflection, judgement, and actions taken. This interaction helped me frame security as a layered engineering problem rather than a single technical trick. After reviewing the response, I decided that the final system should not rely on only one mechanism such as filtered builtins or AST validation. Instead, I adopted a layered design in which the LLM analysis path is separated from the normal API path, database access for analysis uses a read-only account, Python code is screened before execution, SQL is filtered before database access, and execution is isolated in a separate worker process. This was one of the most important uses of GenAI in the project because it directly influenced the final security architecture.

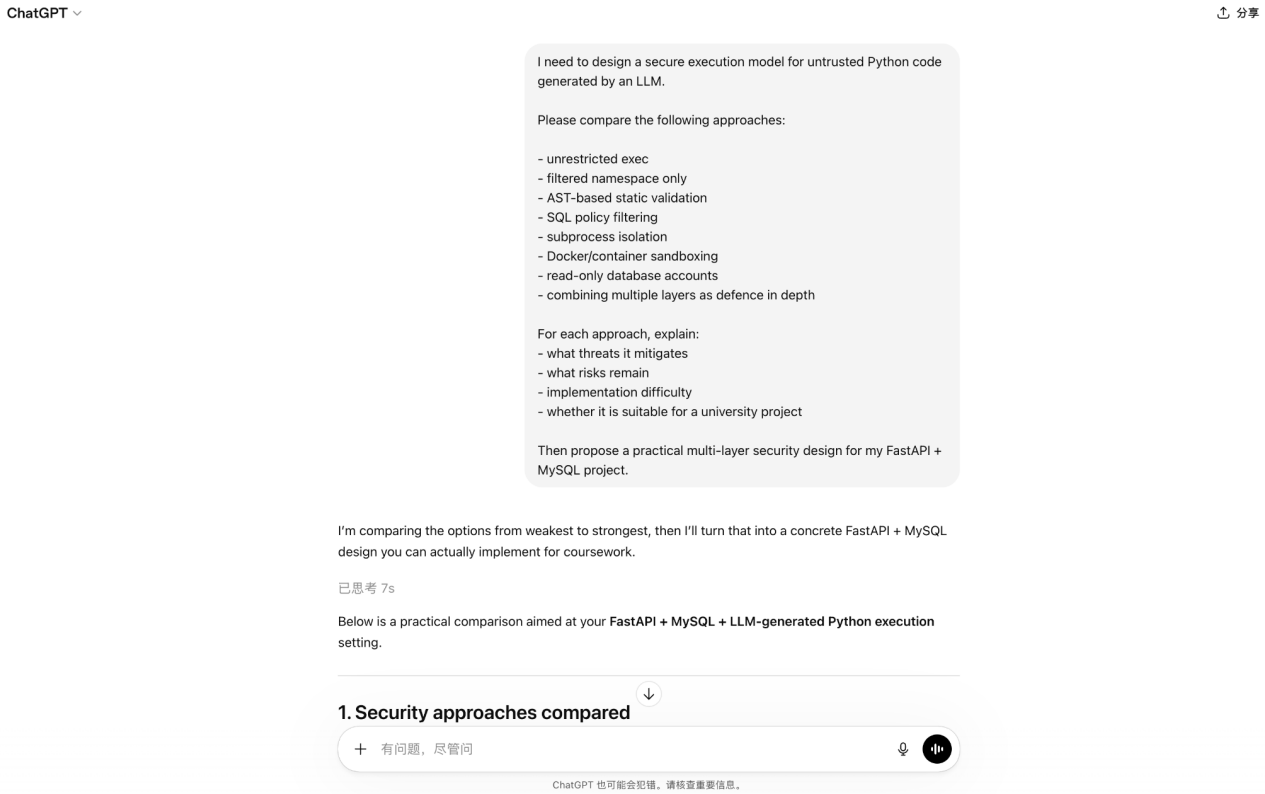

Figure 6: ChatGPT-assisted comparison of layered security strategies for untrusted LLM-generated Python execution.

A3. Nano Banana — academic diagram generation and visual communication

Role in the project. Nano Banana was used to generate the technical diagrams included in the report. Its role was not conceptual decision-making, but *visual communication*: transforming a defined system design into publication-style figures suitable for an academic technical report.

Representative prompt. I provided a detailed prompt describing the desired architecture diagram, including publication-quality academic style, grayscale layout with subtle blue accents, hierarchical top-down structure, exact module names, exact container labels, and explicit connections between the application layer, isolated execution environment, and data layer.

Summary of the response. The model generated a system architecture diagram matching the specified structure and style requirements. Similar prompt patterns were later used to produce the security policy diagram, the sequence-flow diagram, and the testing overview diagram used in the final report.

My reflection, judgement, and actions taken. This interaction shows that GenAI was also used for professional technical presentation, not only for reasoning or coding. I did not accept the visual outputs automatically. After generation, I manually reviewed the diagrams for terminology consistency, alignment with the actual implementation, readability in PDF format, and suitability for a university report. The diagrams were then incorporated into the technical report after layout and consistency checks.

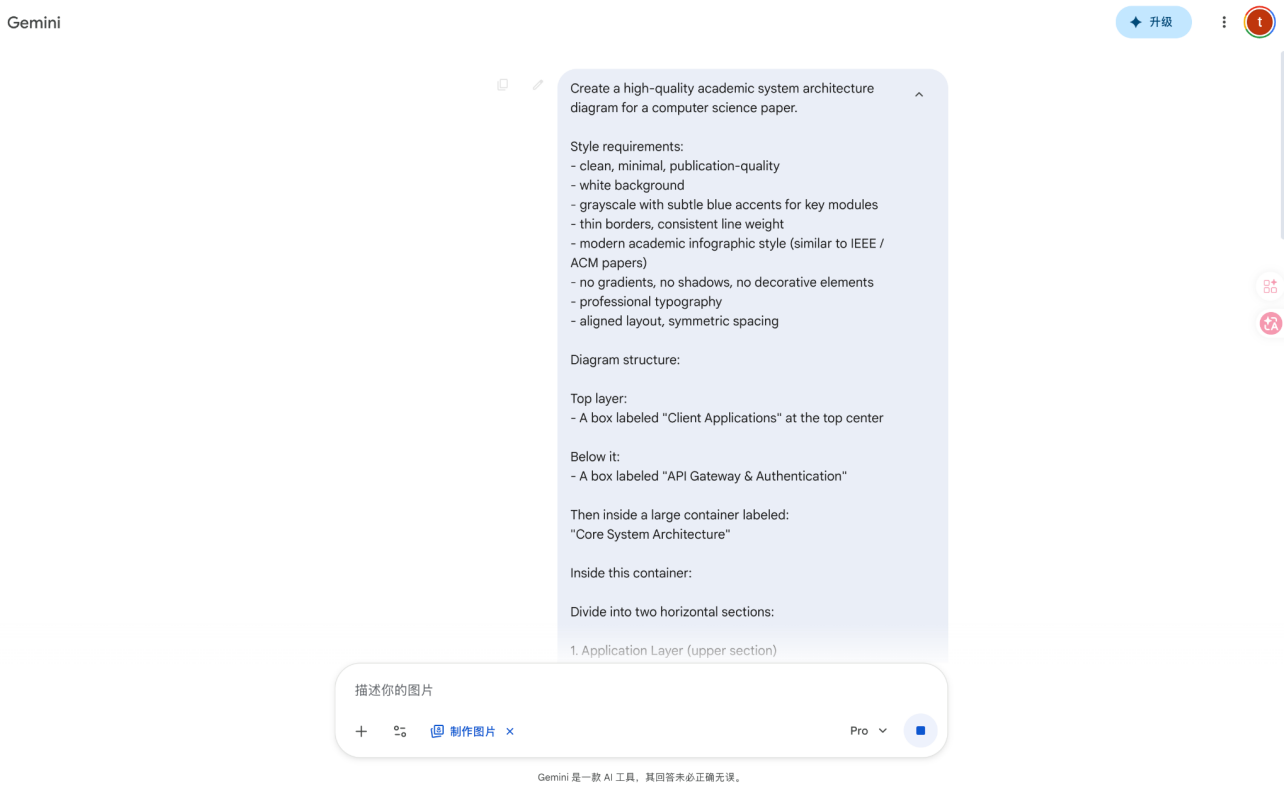

Figure 7: Prompt used to generate the academic system architecture diagram included in the report.

A4. Windsurf — implementation-oriented technical review and hardening feedback

Role in the project. Windsurf was used not only as an implementation-oriented assistant, but also as a *technical reviewer* after most of the system had already been implemented. This is important because it shows that GenAI was used for review, critique, and refinement, not only for code generation.

Representative prompt / context. I asked Windsurf to review the final design of InsightAgent as a FastAPI + MySQL + Docker enterprise analytics API with CRUD endpoints, analytics endpoints, API key authentication, a self-loop LLM agent, `observe_schema` and `run_python_analysis` skills, AST validation, SQL policy filtering, subprocess isolation, dual database accounts, logging, and testing.

Summary of the response. Windsurf examined the implemented codebase and produced a structured technical review covering architecture coherence, security model strengths and weaknesses, skill design scope, testing quality, likely examiner questions, and improvement opportunities. The review was especially useful because it identified concrete weaknesses and viva risks, including possible sandbox bypasses through pre-injected objects, misconfiguration risks around authentication, report-versus-code mismatches, missing schema caching, and weaknesses in some negative tests. The uploaded text excerpt records this detailed review.

My reflection, judgement, and actions taken. This interaction was valuable because it moved beyond "help me write code" and instead acted as a post-implementation audit. After reviewing the response, I used it in three ways: first, to identify which design choices should be emphasised in the report and oral presentation; second, to identify which limitations should be disclosed honestly rather than hidden; and third, to decide which issues were feasible to tighten before submission and which should be recorded as future hardening work. In other words, Windsurf functioned as a critical second reader of the project, helping me improve both the technical explanation and the final presentation of the system.

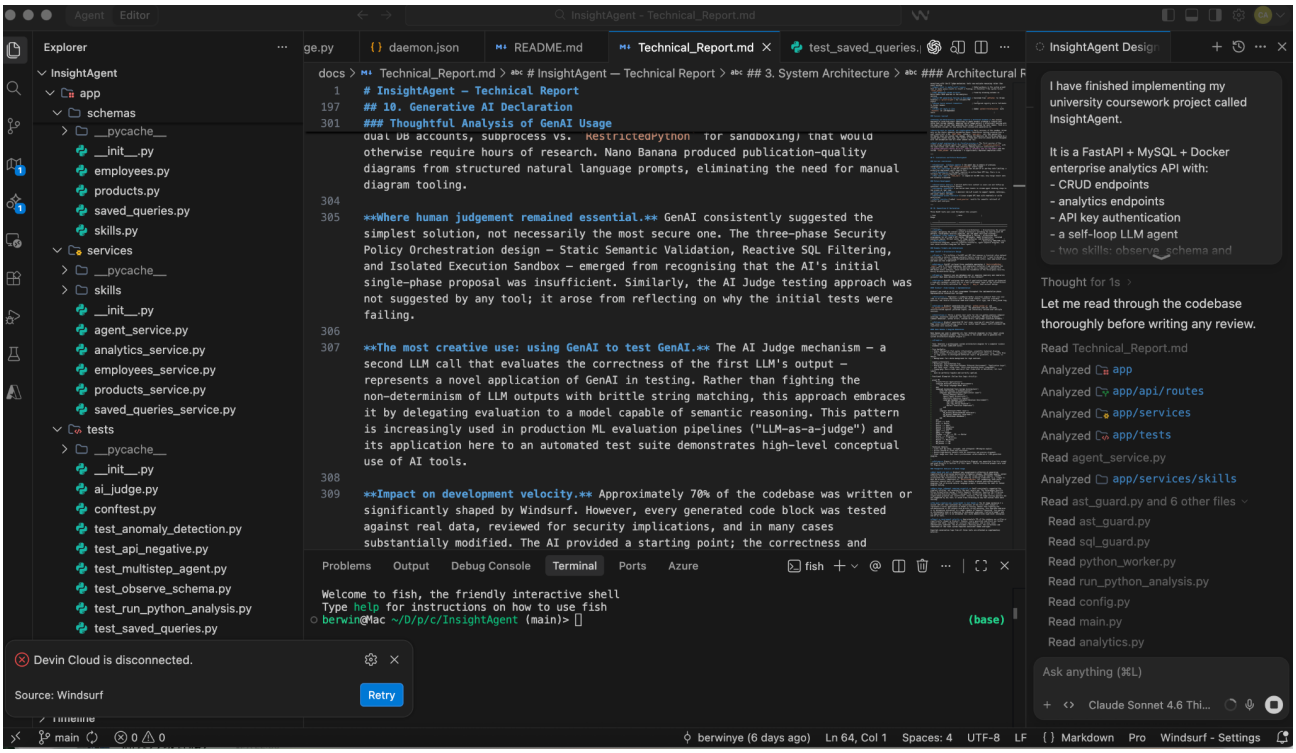

Figure 8: Windsurf-based implementation review and technical critique of the completed InsightAgent codebase.

A5. Overall reflection

Across the project, the three tools contributed at different levels of abstraction. **ChatGPT** was most useful at the architecture and security-design stage, especially for exploring alternatives and comparing high-level trade-offs. **Nano Banana** was most useful for technical visual communication, turning system descriptions into publication-style diagrams. **Windsurf** was most useful during implementation and review, especially for project structure, technical review, and identifying weaknesses that might be questioned in the viva.

The most important point is that GenAI did not replace engineering judgement. In every case, the outputs were treated as proposals rather than authority. My own role as the author was to compare options, reject weak or unsafe suggestions, align the final design with coursework requirements, test the implementation against real behaviour, and explicitly document both strengths and limitations. GenAI therefore functioned as a high-level design partner, implementation assistant, technical reviewer, and visual communication aid rather than as an unsupervised development substitute.